UNIVERSITI PENDIDIKAN SULTAN IDRIS

اونيۏرسيتي ڤنديديقن سلطان ادريس

SULTAN IDRIS EDUCATION UNIVERSITY

Software Verification and Validation (DES3103)

by

Dr. Nor Asiah Mohamad@Razak

Course Learning Outline

1. Briefing Session (Ice Breaking via Whiteboard, Class rules & Pro forma): **M. Teams**
2. Introduction to Software Verification & Validation I & II (V&V)
3. Managing V&V with SQA Integration
4. Quality metrics or attributes in V&V

WEEK 6-MID-TERM TEST (QUIZ): Topic 1 – 3-(CLO1:C2) – 20 MARKS

WEEK 7-PHASE 1: ASSIGNMENT 1-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO3:A5) - 20 MARKS: M. Teams

WEEK 9-PHASE 1: PRESENTATION & REFLECTION ON PHASE 1: M. Teams

5. Software Reviews – Will be covered on Assignments 1 & 2
6. Quality Control: Software Testing - Will be covered in Assignments 1 & 2
7. Human-Computer Interface Testing - Will be covered in Assignments 1 & 2
8. Manual Testing and Automation Testing: Will be covered in Final Assessment

WEEK 11-PHASE 2: ASSIGNMENT 2-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO2&4:P4&C5) - 20 MARKS: M. Teams
WEEK 13-PHASE 2: PRESENTATION & REFLECTION ON PHASE 2: M. Teams

9. Quality Maturity Models
10. Software Problem Analysis and Reporting
11. Quality Culture in doing V&V.
12. Safety and Security in V&V

WEEK 14: REFLECTION ON IMPLEMENTATION OPBL (SOFTWARE TESTERS) IN THE V&V COURSE: e-FORUM

WEEK 16-PHASE 3: FINAL ASSESSMENT – INDIVIDUAL SPECIAL PROJECT BASED ON PHASE 2 (CLO4:C5) (2 WEEKS) - 40 MARKS : e-ASSESSMENT

TOPIC 10

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Software Problem Analysis and Reporting

- Analyzing software failure reports
- Problem tracking and defect analysis

Software Problem Analysis and Reporting - Process Model for Requirement Change Management (RCM)

Software Project Failure

One of many reasons of software project failure is requirement changes. Different requirement changes come up during different phases of software development.
Managing these changes throughout the software life cycle, especially in later development phases is a challenging task. Mismanaged requirement changes can also lead to the failure of the project.

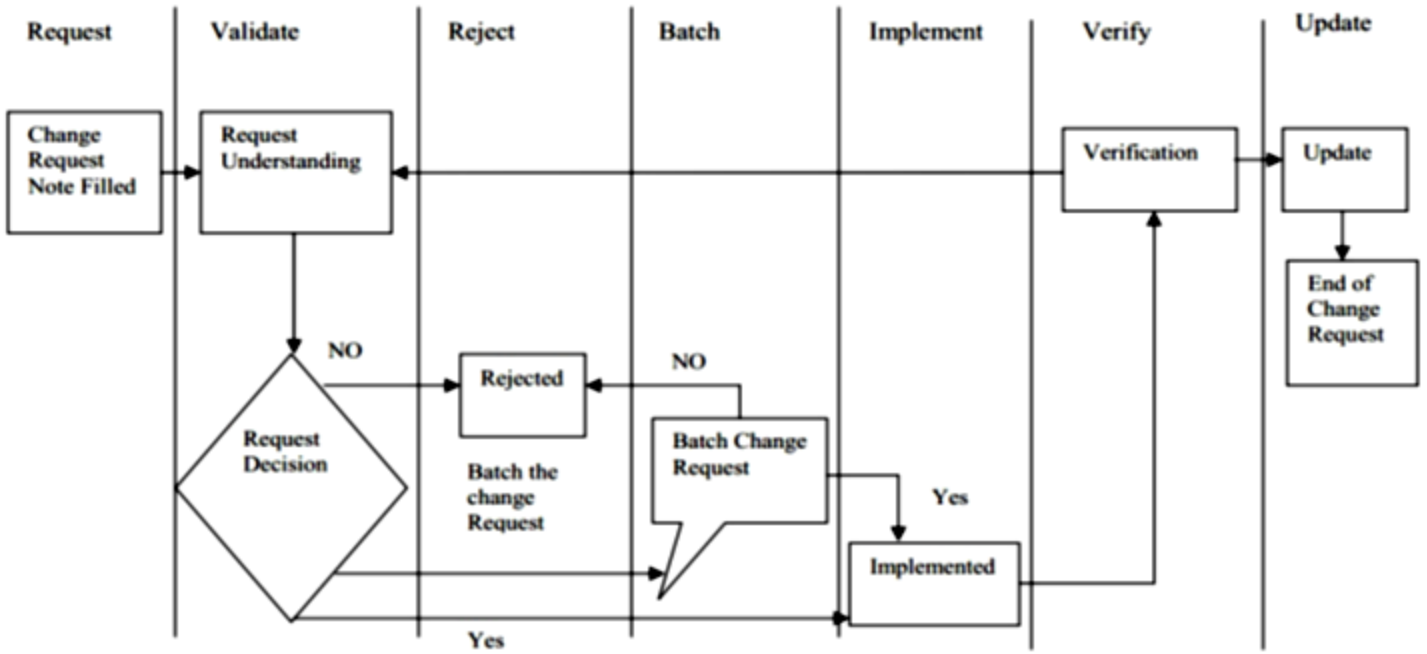


Figure 1. Process Model for RCM

- Arif Ali Khan et al. proposed seven (7) stages Change management model comprises of the following phases, Request, Validation, Rejection, Batch Change, Implementation, Verification, and Update.
- In the first phase, a change request is generated, and after that, it is forwarded to the validation phase where it is decided whether the change request should be implemented, batched, or rejected.
- In case of implementation, the implemented change is again sent to the verification phase to make sure that the change applied is working properly or not, then it is recorded in the documentation. This model is depicted in Figure 1.

Software Problem Analysis and Reporting

Spiral Model for Software Change Management

- As demonstrated in Figure 1, this spiral model is comprised of four cycles or basic steps. During the initial cycle; several modifications/changes are requested comprises of correction of errors in the existing system or inclusion of additional features or functionalities of the system.
- In the conclusion of initial cycle, it is decided by the problem holder whether the change is necessary or not and if it is important, then how it is to be accommodated.
- The next stage of the spiral model takes place when the desired change is seen from non-technical perspective.
- The third cycle is planning phase of the spiral model. In this stage a strategy is designed through which change implementation is accomplished.

Implementation of changes and technical solution verifications are the activities of change implementation stage. It is also the last phase of this change request cycle. Each update concerning the changes are recorded after the completion of change implementation. Being a generic model, it is employed to handle the requirements change in both; maintenance and the development stages.

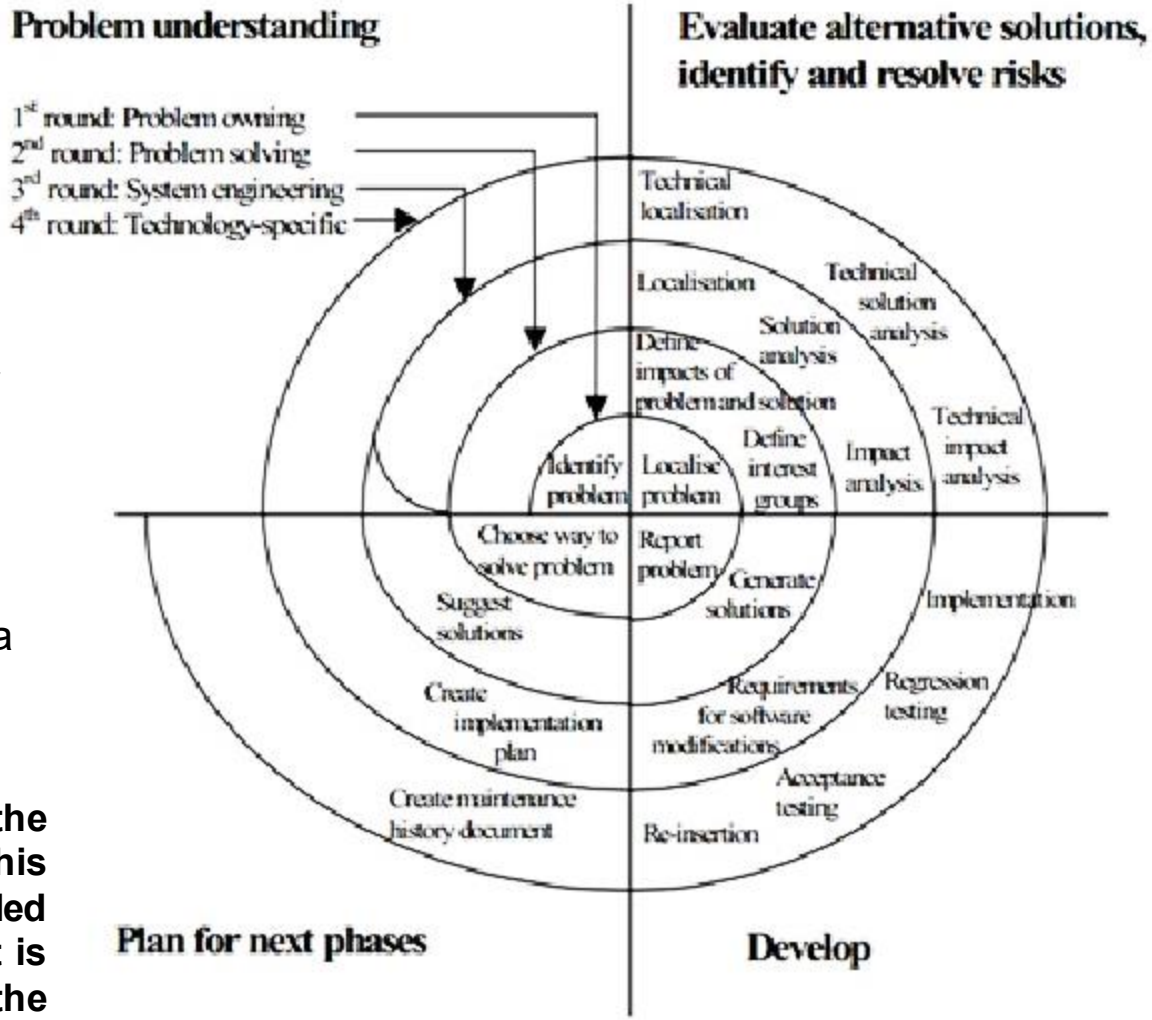


Figure 2: Spiral Model for Software Change Management

Analyzing software failure reports

Failure analysis is the process of collecting and analyzing data to determine a cause of a failure and how to prevent it from recurring. It is an important discipline in many branches of manufacturing industry, such as the electronics, where it is a vital tool used in the development of new products and for the improvement of existing products. Failure analysis is especially important in manufacturing and field use of safety-critical and mission-critical equipment.

Failure analysis may be applied to both products and processes.
Failure analysis may be conducted at the design stage and at the field use stage of a product life cycle.

There are several different failure analysis methods and tools:

Figure 3. FRACAS (Failure Reporting, Analysis, and Corrective Action)

Failure Data Collection

Failure analysis of safety-critical and mission-critical equipment at the field use stage requires failure data collection and statistical analysis. To accomplish this goal a special software system called FRACAS (Failure Reporting, Analysis and Corrective Actions System) should be used. Collected data are subject to statistical analysis. Such system also gives real-time added value for the organization and acts like fleet management system, safety management system, workflow system with alerts and escalation and more.

Collected statistics and obtained field failure rates could be used in all failure analysis methods mentioned below.

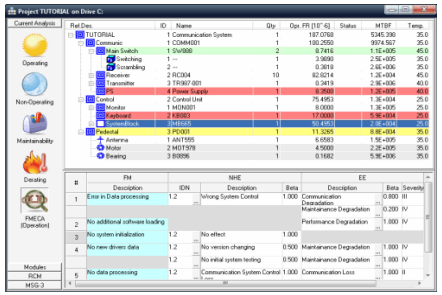
Software Problem Analysis and Reporting - Analyzing software failure reports

Other Analyzing software failure reports

FMECA: (Failure Mode, Effects and Criticality Analysis)

Hardware FMEA and FMECA (Failure Mode, Effects and Criticality Analysis) is a continuation of system reliability analysis. FMECA analysis will be performed for all designed/ manufactured/ acquired systems, especially if they are mission or safety critical. FMECA includes failure analysis, criticality analysis and testability analysis. FMECA analyzes different failure modes and their effects on the system, classifies and prioritizes end effects level of importance based on failure rate and severity of the failure effects.

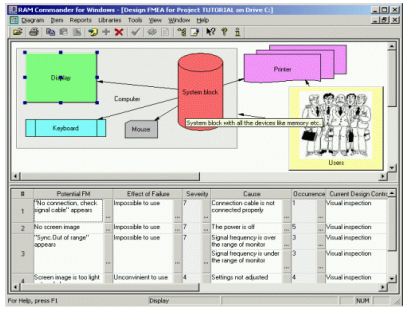
RAM Commander FMECA Software Tool



FMEA - (Failure Mode and Effects Analysis)

Potential FMEA (Failure Mode and Effects Analysis) is analytical technique utilized as a mean to assure that, to the extent possible, potential failure modes and their associated causes/mechanisms have been considered and addressed. Corrective actions are suggested and selected for implementation and control plan is formed as a part of the procedure. Sometimes its called "automotive" or "AIAG" (Automotive Industry Action Group) FMEA.

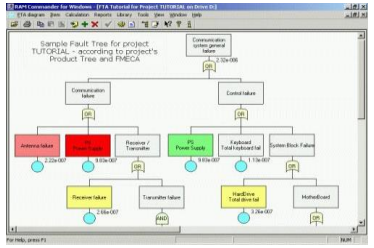
RAM Commander FMEA Software Tool



Fault Tree Analysis (FTA)

Fault Trees are one of the most widely used methods in system reliability and failure probability analysis. A Fault Tree is a graphical representation of events in a hierarchical, tree-like structure. It is used to determine various combinations of hardware, software, and human failures that could result in a specified risk or system failure.

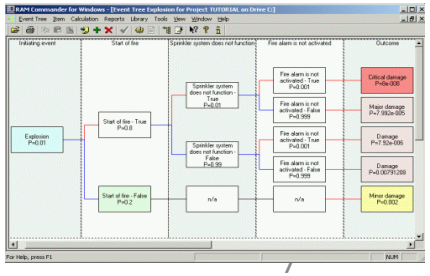
RAM Commander Fault Tree Analysis Software Tool



Event Tree Analysis (ETA)

It is an inductive failure analysis performed to determine the consequences of a single failure for the overall system risk or reliability. Event Tree Analysis uses similar logic and mathematics as Fault Tree Analysis, but the approach is different

RAM Commander ETA Software Tool



Software Problem Analysis and Reporting - Problem tracking and defect analysis

Issue tracking systems

Defect details: Defect ID | Detected date | Status | Customer ref | Reported by | Environment (QA, Dev, Production) | Assigned to

What is an Issue Tracking System (ITS)?

Issue tracking systems, commonly referred to as ITS, are software applications that provide a ticketing system to record and follow the progress of every issue identified by a computer user until the issue is resolved.
An issue can be anything from a bug report to customer question to development inquiry. Issues are often confused with bugs because all bugs are issues, but not all issues are bugs. A typical bug is a defect in the codebase, and a single bug can manifest in many different ways.
When ten different customers report ten different issues caused by the same bug, ten tickets are opened in an ITS and are tracked until they are resolved, usually by applying a patch and verifying with the affected customers that they're no longer experiencing any problems.

Defect Tracking Template for Excel

Top 6 Best Issue Tracking Systems in 2020

1. JIRA
2. Backlog
3. Trac
4. Redmine
5. WebIssues
6. Asana

Features of Issue Tracking Systems

Each issue in an ITS typically has several details associated with it. Some issues may have greater urgency than others, perhaps because they affect the most customers or because they represent a serious roadblock that needs to be solved for the project to continue smoothly.
Likewise, issues can also have low or zero urgency to indicate that they should be resolved as time permits.
Other details include everything from the customer experiencing the issue to detailed descriptions of the issue being experienced to attempted solutions and other relevant information.

Most ITS solutions also make it possible to **assign issues** to various persons in charge, **monitor** how they are being handled and how much time is being spent on them, **ensure the compliance** with internal workflows, perform **statistical analysis**, and automatically generate tickets based on customer inquiries, just to give a few examples.
Compared with more traditional issue tracking methods, modern ITS solutions provide accountability and the necessary measures to ensure that issues are resolved in a satisfactory manner, and they are often integrated with other project development tools.
We have selected six of **the best issues tracking systems**, making sure to include both popular powerhouse solutions as wells as smaller open-source projects created to satisfy the needs of independent developers and budget-minded agencies alike.

Software Problem Analysis and Reporting - Problem tracking and defect analysis

Project Defect Template

Defect details: Defect ID | Detected date | Status | Customer ref | Reported by | Environment (QA, Dev, Production) | Assigned to

Defect or issues impact

The impact ratings and descriptions below are examples, which can be easily changed to suit your project.

User impact

- Critical - issues or bugs that prevent the system or product from being usuable. They must be fixed in order to launch/go live.
- High - defects that need to be resolved, but that have a workaround.
- Medium - issues that are inconvenient, but don't prevent the full use of the system or product. They may nor may not be fixed.
- Low - Issues (often ideas that would be nice to have) that have no real impact on the users. Will probably not be implemented/fixed unless they become more critical.

Business impact

- Critical - issues or bugs that mean that the product or system will not meet a key business objective of the project.
- High - defects that have a work around but need to be resolved because they will impact whether the product will meet its business objective(s).
- Medium - bugs / issues that might reduce the ability of the system to meet business objectives. Consider fixing in a later phase or in an upgrade.
- Low - Niggles or ideas not related to the business objectives. Ideas or enhancement suggestions can be logged for future projects.

Summary

A brief summary or title for the issue.

Defect / issue description

Give a detailed description of the defect, ideally with the steps to replicate (the steps to follow to cause the defect).

Customer facing update

Enter the latest status of the issue and next steps.

Internal update

This is an optional space to enter more details for your project team. For example, code, reference numbers, or thoughts about the cause that are not ready to be shared with the customer.

DEFECT / ISSUE TRACKER

Status	Count		User Impact	Business impact
1-New	1	1-Critical	0	1
2-Open	4	2-High	2	2
3-Closed	0	3-Medium	3	0
TOTALS	5	4-Low	0	2
			5	5

Defect ID	Detected Date	Status	HIML User impact	HIML Business impact	Customer ref	Subject	Reported by	Defect / issue description	Environment	Customer facing update
1	31/01/2021	1-New	2-High	2-High	438	CRM connector: Email address gives error when saving.	Customer test team	Search for a relation (with email address) in the connector. Select relation. Confirm your choice. Enter empty fields. Click save and close Expected: Relation will be stored and closed Observed behavior: Error message	QA	With Engineer for review.
2	15/01/2021	2-Open	3-Medium	4-Low	441	Need button to clear all fields.	Customer test team	The connector tab contains 8 search fields. I'm missing a button here to clear all fields for a new search.	Dev	New requirement. Workaround is to manually clear the fields. Engineer is assessing the effort.
3	30/12/2020	2-Open	3-Medium	2-High	442	Nationality is not given correctly.	Customer test team	Insured person (policy member) - nationality is not coming through connector.	QA	With Engineer for review.

END

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Thank You…
Terima kasih..